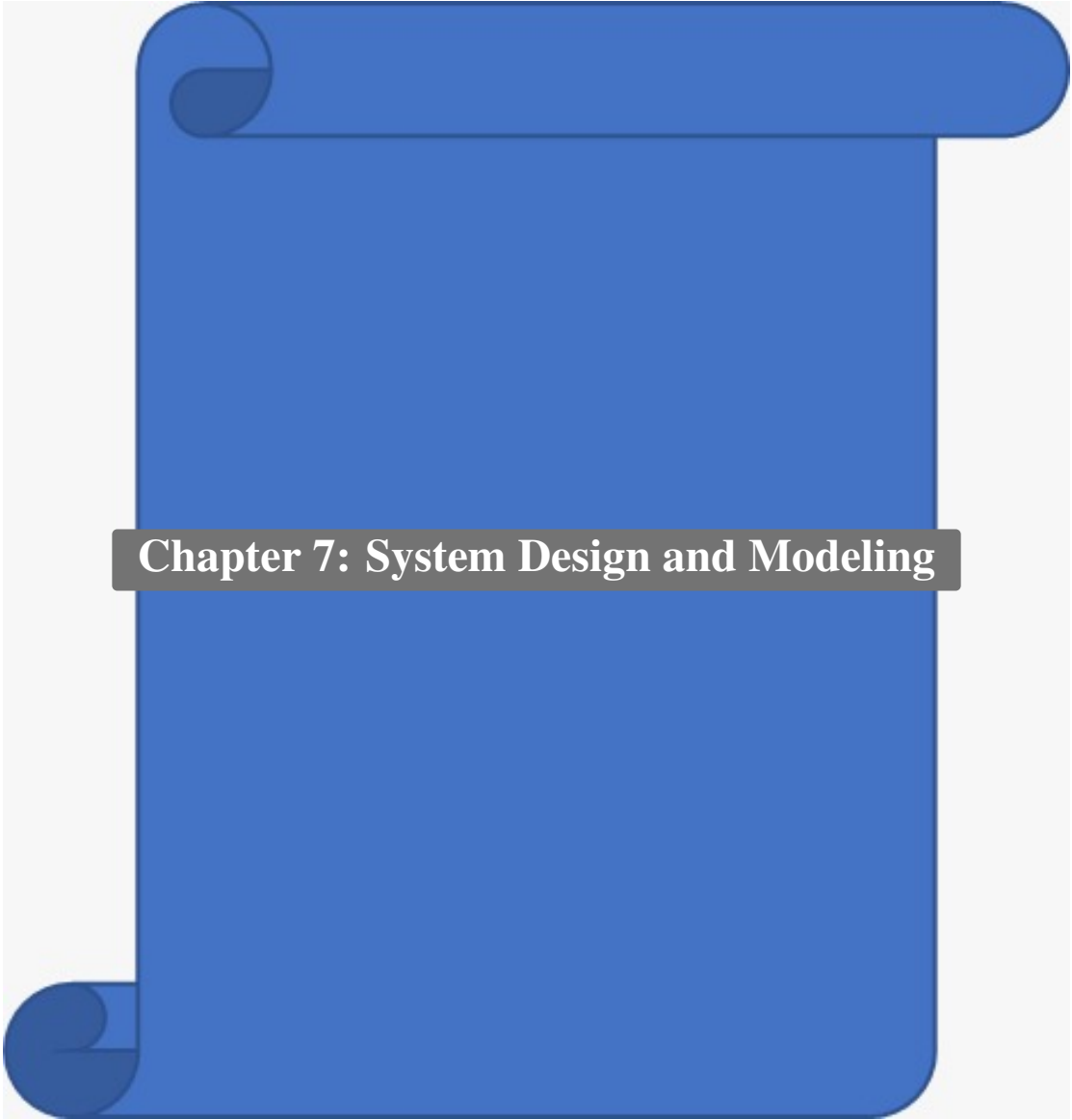




Chapter 7: System Design and Modeling

7.1 Gantt Chart

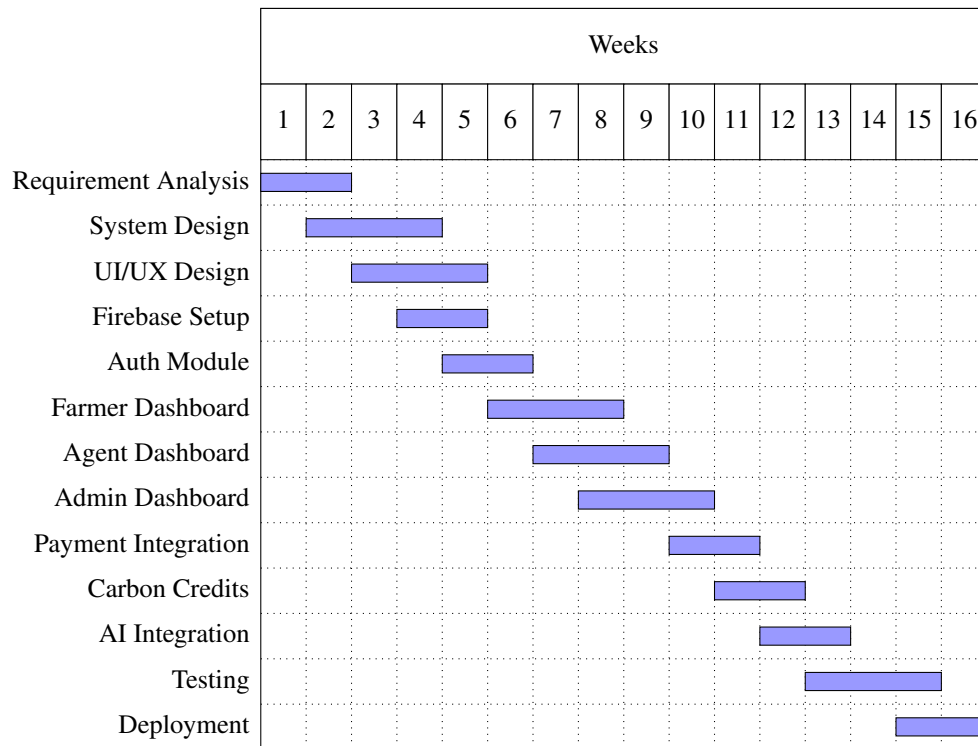


Figure 7.1: Project Development Gantt Chart

7.2 DFD (Data Flow Diagram)

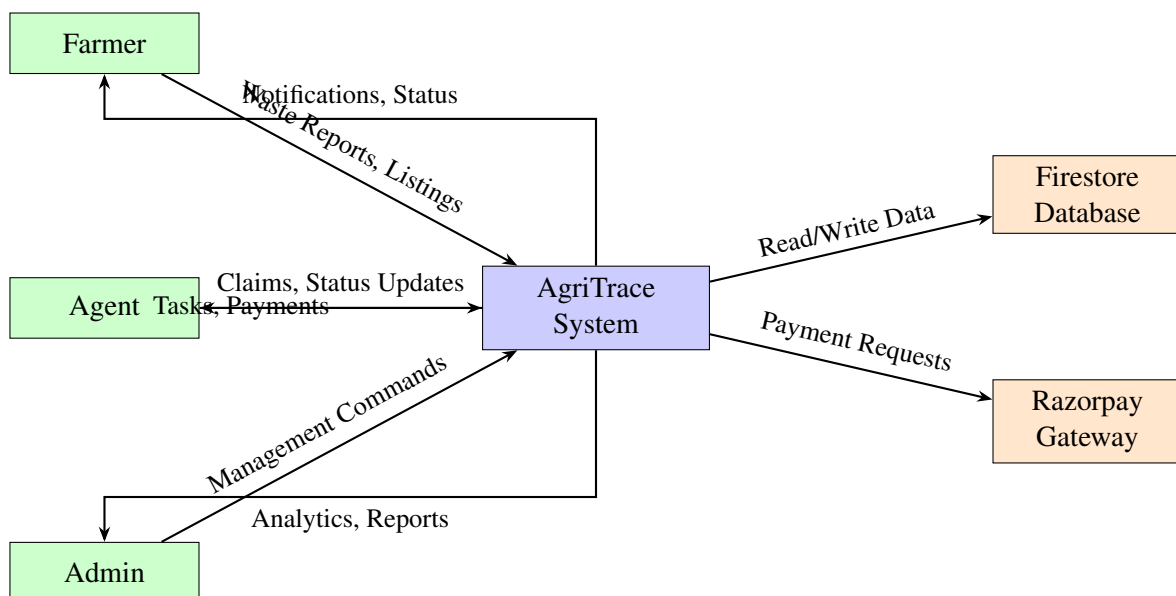


Figure 7.2: Level 0 Data Flow Diagram

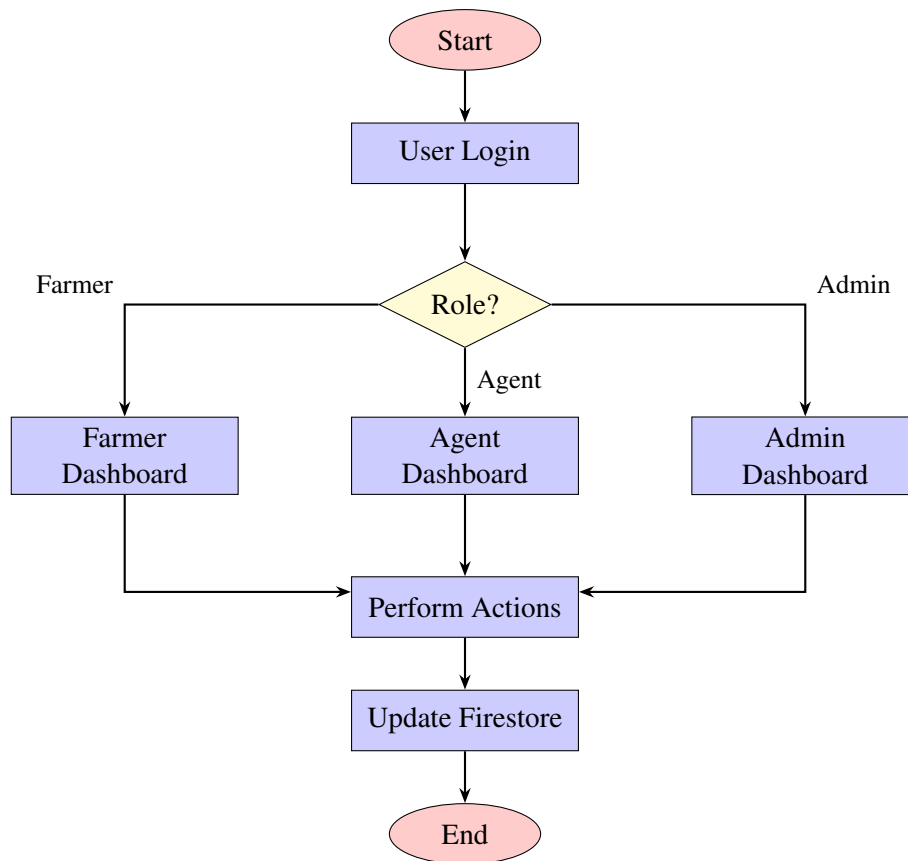


Figure 7.3: Application Flow Diagram

7.3 Flow Diagram

7.4 Use Case Diagram

7.5 Activity Diagram

7.6 ER Diagram

The ER diagram illustrates the six core entities of the AgriTrace system and their relationships. The User entity is central, connecting to Listings (creation relationship), CarbonCredits (earning relationship), and Notifications (receiving relationship). Listings generate Orders, which in turn require Payments. These relationships are implemented as Firestore collections with document references.

7.7 Firestore Collection Schema

The following table details the Firestore collections, their fields, data types, and descriptions:

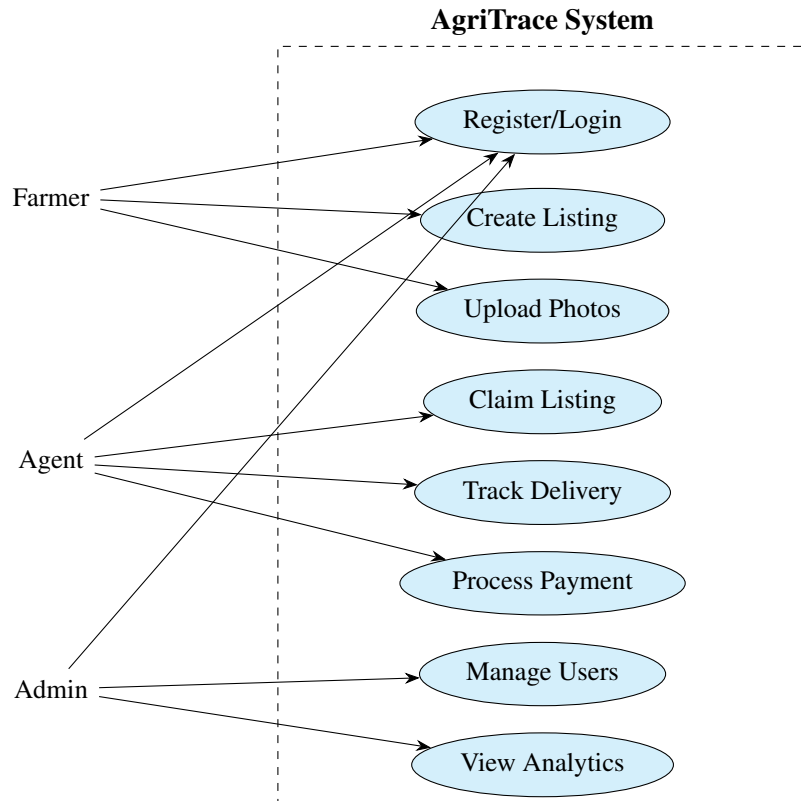


Figure 7.4: Use Case Diagram

Table 7.1: Firestore Collection Schema – Users

Collection	Field	Type	Description
users	uid	string	Firestore Auth UID (document ID)
users	email	string	User's email address
users	name	string	Display name
users	role	string	User role: farmer, agent, or admin
users	phone	string	Contact phone number
users	verified	boolean	Email verification status
users	profilePhoto	string	URL to profile image
users	location	map	Latitude, longitude, address
users	createdAt	timestamp	Account creation timestamp

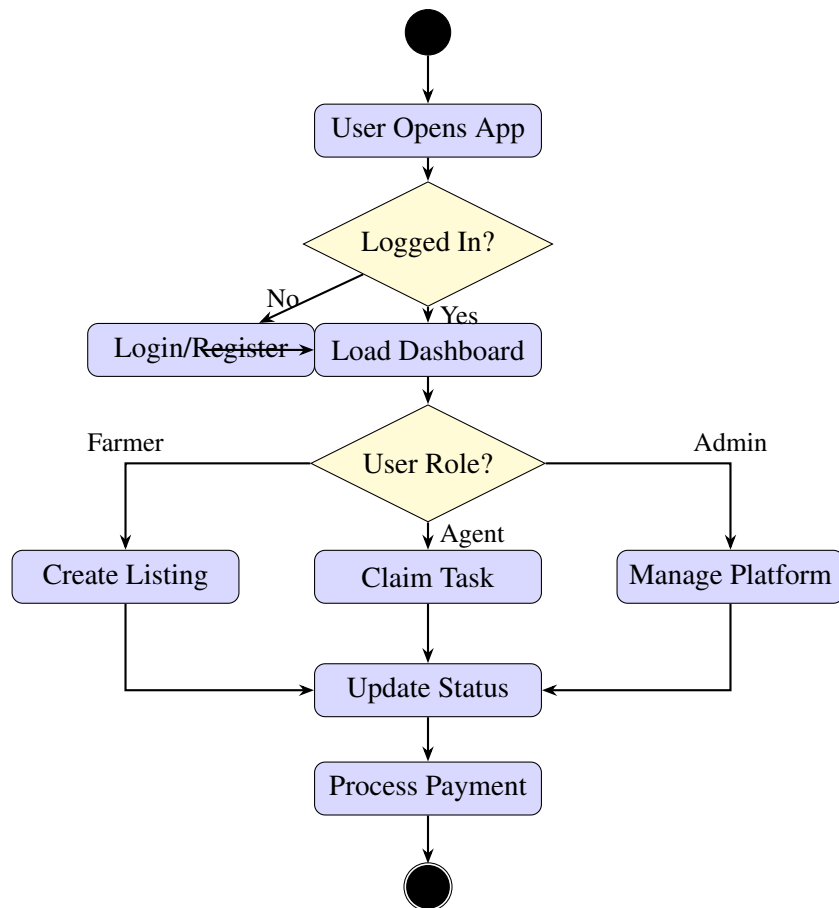


Figure 7.5: Activity Diagram

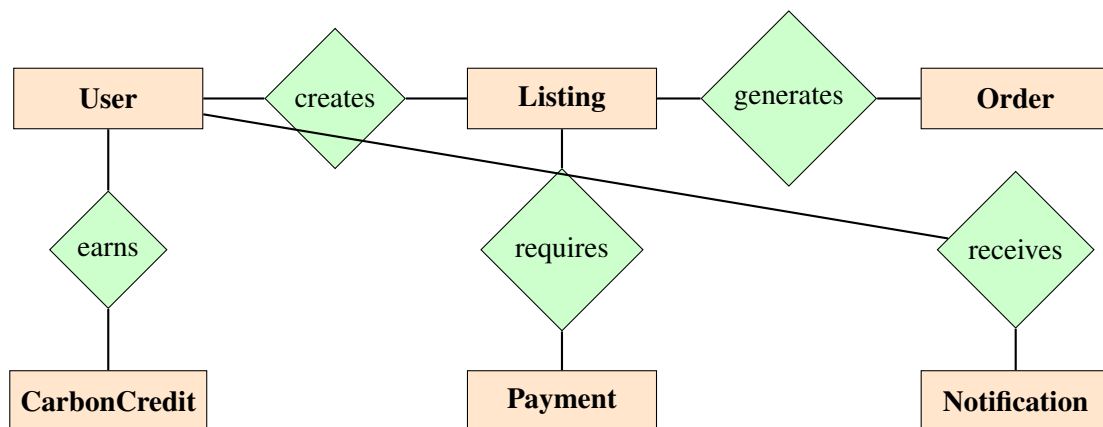


Figure 7.6: Entity-Relationship Diagram

Table 7.2: Firestore Collection Schema – Listings

Collection	Field	Type	Description
listings	id	string	Auto-generated document ID
listings	title	string	Listing title/description

Collection	Field	Type	Description
listings	category	string	Waste type (Rice Husk, Wheat Stubble, etc.)
listings	quantity	number	Quantity in metric tonnes
listings	price	number	Price per unit in INR
listings	description	string	Detailed listing description
listings	photos	array	Array of Cloudinary image URLs
listings	sellerId	string	Reference to seller's user document
listings	sellerEmail	string	Seller's email for quick lookup
listings	status	string	OPEN, ASSIGNED, IN_TRANSIT, DELIVERED, RECYCLED, CANCELLED
listings	assignedAgentId	string	Reference to assigned agent
listings	assignedAgentEmail	string	Agent's email for display
listings	location	map	GPS coordinates and address
listings	assignedAgentEmail	string	Email of the assigned collection agent
listings	paymentStatus	string	Status of payment to farmer (PENDING, PAID)
listings	orderId	string	Reference to the recorded order document
listings	createdAt	timestamp	Listing creation time
listings	updatedAt	timestamp	Last status update time

Table 7.3: Firestore Collection Schema – Orders and Notifications

Collection	Field	Type	Description
orders	id	string	Auto-generated document ID
orders	listingId	string	Reference to listing document

Collection	Field	Type	Description
orders	buyerId	string	Reference to buyer's user document
orders	sellerId	string	Reference to seller's user document
orders	price	number	Transaction amount in INR
orders	status	string	Pending, Accepted, Rejected, Completed
orders	razorpayOrderId	string	Razorpay payment order reference
orders	createdAt	timestamp	Order creation time
notifications	id	string	Auto-generated document ID
notifications	userId	string	Target user for this notification
notifications	title	string	Notification heading
notifications	message	string	Notification body text
notifications	type	string	info, success, warning, error
notifications	read	boolean	Whether user has read the notification
notifications	link	string	Optional deep link to related content
notifications	createdAt	timestamp	Notification creation time

7.8 Component Hierarchy

The AgriTrace application follows a hierarchical component structure aligned with the Next.js App Router pattern. The following describes the component tree:

- **Root Layout** (`app/layout.tsx`)
 - `AuthProvider` – Wraps the entire app with authentication context
 - `Toaster` – Global toast notification component
 - **Landing Page** (`app/page.tsx`) – Public-facing homepage
 - **Login Page** (`app/login/page.tsx`)
 - * `LoginForm` – Email/password input with validation

- * TestimonialCarousel – Rotating user testimonials
- **Signup Page** (app/signup/page.tsx)
 - * SignupForm – Registration with role selection
- **Dashboard Page** (app/dashboard/page.tsx)
 - * FarmerDashboard – Farmers-only content
 - StatCard – Statistics display cards
 - PhotoUploader – Image upload component
 - LocationPicker – GPS coordinate capture
 - PaymentButton – Razorpay integration
 - ListingCard – Individual listing display
 - * AgentDashboard – Agents-only content
 - StatCard – Statistics display cards
 - TaskCard – Available/claimed task cards
- **Admin Page** (app/admin/page.tsx)
 - * AdminStatCards – Platform-wide statistics
 - * UserManagement – User list with role editing
 - * ListingManagement – All listings with recycling controls
 - * AnalyticsCharts – Recharts-based data visualizations
- **Shared Components** (components/ui/)
 - * Button, Dialog, Card, Badge – Radix UI primitives
 - * Select, Input, Label – Form elements
 - * Toast, Tabs, Sheet – Interactive elements

7.9 Security Architecture

The security architecture of AgriTrace operates at multiple levels:

1. **Transport Layer:** All communication between the client and Firebase services is encrypted using TLS 1.3. The application is served over HTTPS, ensuring data confidentiality in transit.
2. **Authentication Layer:** Firebase Authentication manages user identity with industry-standard practices including secure password hashing (bcrypt), session token management, and automatic token refresh. Password reset flows use time-limited, single-use tokens sent via email.
3. **Authorization Layer:** Firestore Security Rules enforce role-based access control (RBAC) at the database level. Rules check the authenticated user's UID and role before allowing read/write operations. Key rules include:

- Users can only modify their own profile documents
 - Listings can only be deleted by their creators
 - Admin-only operations check for the admin role in the user's profile document
 - Delete operations on critical collections (users, orders) are universally denied
4. **Application Layer:** Client-side route guards redirect unauthenticated users to the login page. Role-based component rendering ensures that UI elements are only shown to authorized users. API routes validate authentication headers before processing requests.
 5. **Payment Security:** Razorpay credentials are stored as server-side environment variables, never exposed to the client. Payment verification uses cryptographic signature matching to prevent amount tampering. The payment flow creates server-side orders, ensuring the payment amount is controlled by the server.